

Practical-5: Inter-Process Communication Using Pipes

Objective

1. Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.
2. Develop a program that executes the equivalent of the shell command `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

Part 1: Producer-Consumer Using Anonymous Pipe

Description

An anonymous pipe is used for communication between a parent and child process.

The parent process acts as the producer and generates integer data.

The child process acts as the consumer and reads the data from the pipe.

The pipe provides one-way communication between the processes.

System Calls Used

- `pipe()` - Creates the communication channel.
- `fork()` - Creates the child process.
- `write()` - Producer writes data into the pipe.
- `read()` - Consumer reads data from the pipe.
- `close()` - Closes unused pipe ends.
- `wait()` - Parent waits for the child process.

Compilation

```
gcc producer_consumer.c -o producer_consumer
```

```
#include <stdio.h> #include <stdlib.h> #include <unistd.h> #include <sys/wait.h>
#include <string.h> #include <time.h>
```

```
int main() { int pipefd[2]; pid_t pid;
```

```

char buffer[100];
int data[] = {10, 20, 30, 40, 50};
int n = sizeof(data) / sizeof(data[0]);

if (pipe(pipefd) == -1)
{
    perror("pipe");
    return 1;
}

pid = fork();

if (pid < 0)
{
    perror("fork");
    return 1;
}

if (pid > 0)
{
    /* Parent - Producer */
    close(pipefd[0]);

    printf("Producer (Parent): Generating data...\n");

    for (int i = 0; i < n; i++)
    {
        int start = clock();

        write(pipefd[1], &data[i], sizeof(data[i]));

        int end = clock();

        printf("Produced: %d | Communication time: %d clock\n",
            data[i], end - start);
    }

    close(pipefd[1]);

    wait(NULL);
}

```

```

        printf("Producer: Finished.\n");
    }
    else
    {
        /* Child - Consumer */
        close(pipefd[1]);

        int value;

        printf("Consumer (Child): Waiting for data...\n");

        while (read(pipefd[0], &value, sizeof(value)) > 0)
        {
            printf("Consumed: %d\n", value);
        }

        close(pipefd[0]);

        printf("Consumer: Finished.\n");
    }

    return 0;

}

#include <stdio.h> #include <stdlib.h> #include <unistd.h> #include <sys/wait.h>

int main(){ int pipefd[2]; pid_t pid1, pid2;

if (pipe(pipefd) == -1)
{
    perror("pipe");
    return 1;
}

pid1 = fork();

if (pid1 < 0)
{
    perror("fork");
    return 1;
}

```

```

if (pid1 == 0)
{
    /* First child: ls -l */
    close(pipefd[0]);

    dup2(pipefd[1], STDOUT_FILENO);

    close(pipefd[1]);

    execlp("ls", "ls", "-l", NULL);

    perror("execlp ls");
    exit(1);
}

pid2 = fork();

if (pid2 < 0)
{
    perror("fork");
    return 1;
}

if (pid2 == 0)
{
    /* Second child: grep ".c" */
    close(pipefd[1]);

    dup2(pipefd[0], STDIN_FILENO);

    close(pipefd[0]);

    execlp("grep", "grep", ".c", NULL);

    perror("execlp grep");
    exit(1);
}

/* Parent */
close(pipefd[0]);
close(pipefd[1]);

```

```
waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

printf("Pipeline execution completed.\n");

return 0;

}
```

Output -

PRACTICAL-05

INTER-PROCESS COMMUNICATION USING PIPES

=====

PART 1: PRODUCER-CONSUMER USING ANONYMOUS PIPE

=====

Compilation:

```
gcc producer_consumer.c -o producer_consumer
```

Execution:

```
./producer_consumer
```

Output:

Producer (Parent): Generating data...

Produced: 10 | Communication time: 2 clock ticks

Produced: 20 | Communication time: 1 clock ticks

Produced: 30 | Communication time: 1 clock ticks

Produced: 40 | Communication time: 1 clock ticks

Produced: 50 | Communication time: 1 clock ticks

Consumer (Child): Waiting for data...

Consumed: 10

Consumed: 20

Consumed: 30

Consumed: 40

Consumed: 50

Consumer: Finished.

Producer: Finished.

OBSERVATION:

The parent process acts as the producer and writes data
into the anonymous pipe.

The child process acts as the consumer and reads the data from the pipe.

The communication time was measured using clock ticks.

The exact values may vary depending on system load.

COMMUNICATION FLOW:

Parent

|

| write()

v

Pipe

|

| read()

v

Child

PART 2: `ls -l | grep ".c"`

=====

Compilation:

```
gcc pipe_ls_grep.c -o pipe_ls_grep
```

Execution:

```
./pipe_ls_grep
```

Output:

```
-rw-r--r-- 1 root root ... pipe_ls_grep.c
```

```
-rw-r--r-- 1 root root ... producer_consumer.c
```

Pipeline execution completed.

OBSERVATION:

The first child process executes `ls -l`.

Its standard output is redirected to the pipe using `dup2()`.

The second child process executes `grep ".c"`.

Its standard input is redirected from the pipe using dup2().

The parent waits for both child processes using waitpid().

SYSTEM CALLS USED:

fork() - Creates child processes.

pipe() - Creates an anonymous pipe.

dup2() - Redirects standard input/output.

exec() - Executes ls and grep.

waitpid() - Waits for child processes.

PIPELINE:

ls -l

|

| pipe

v

grep ".c"

|

v

Output

CONCLUSION:

Both inter-process communication experiments were successfully implemented using Linux system calls.

The producer-consumer program demonstrated communication between parent and child processes using an anonymous pipe.

The second program successfully implemented the equivalent of the shell command:

```
ls -l | grep ".c"
```

using `fork()`, `pipe()`, `dup2()`, and `exec()`.